

MEDS Lab - Module 4

Section 11 Report: Finite State Machines

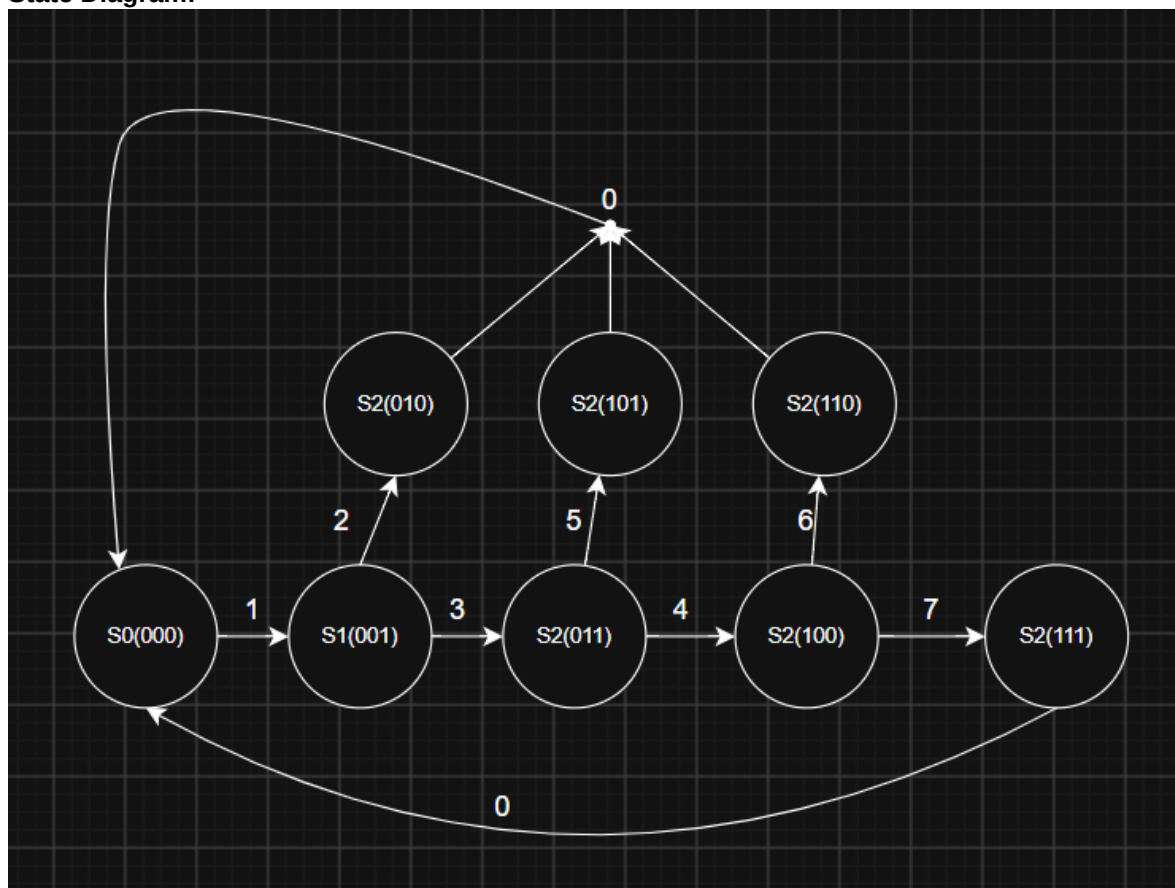
Name: Muhammad Kaleem Faryad

University: UET Lahore

Task 14: Custom-Sequence Counter with T Flip-Flops (State Machine View)

1. EDA Playground Link: <https://edaplayground.com/x/Csnf>

2. State Diagram:

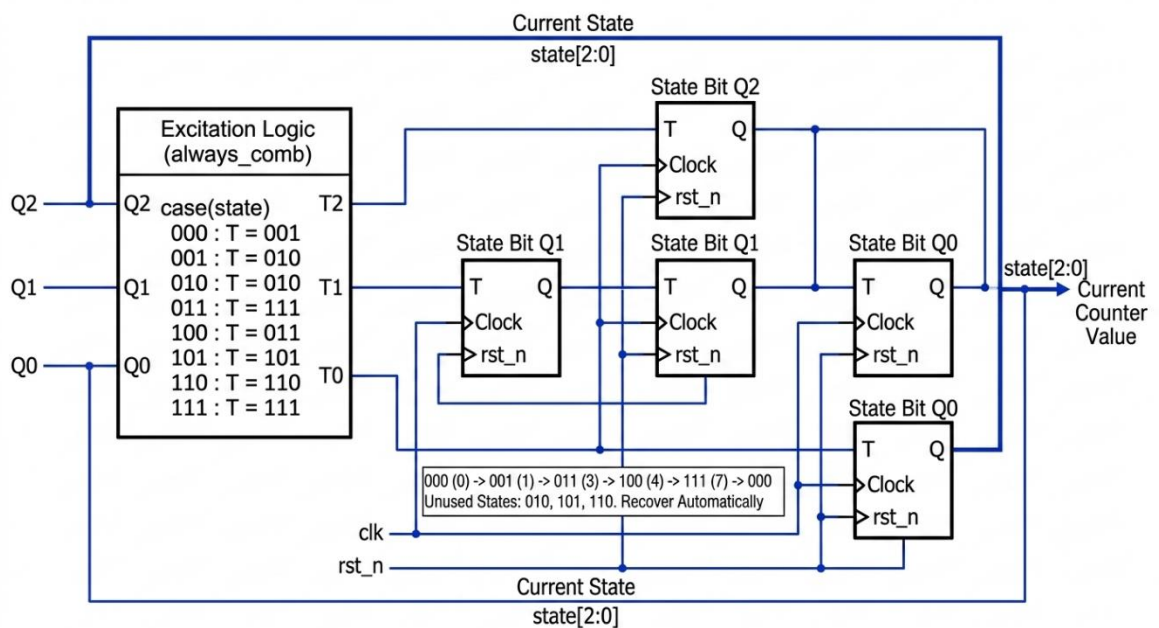


3. State table:

Current State	Next State	T2	T1	T0
000	001	0	0	1
001	011	0	1	0
010	000	0	1	0
011	100	1	1	1

Current State	Next State	T2	T1	T0
100	111	0	1	1
101	000	1	0	1
110	000	1	1	0
111	000	1	1	1

4. Circuit Diagram:



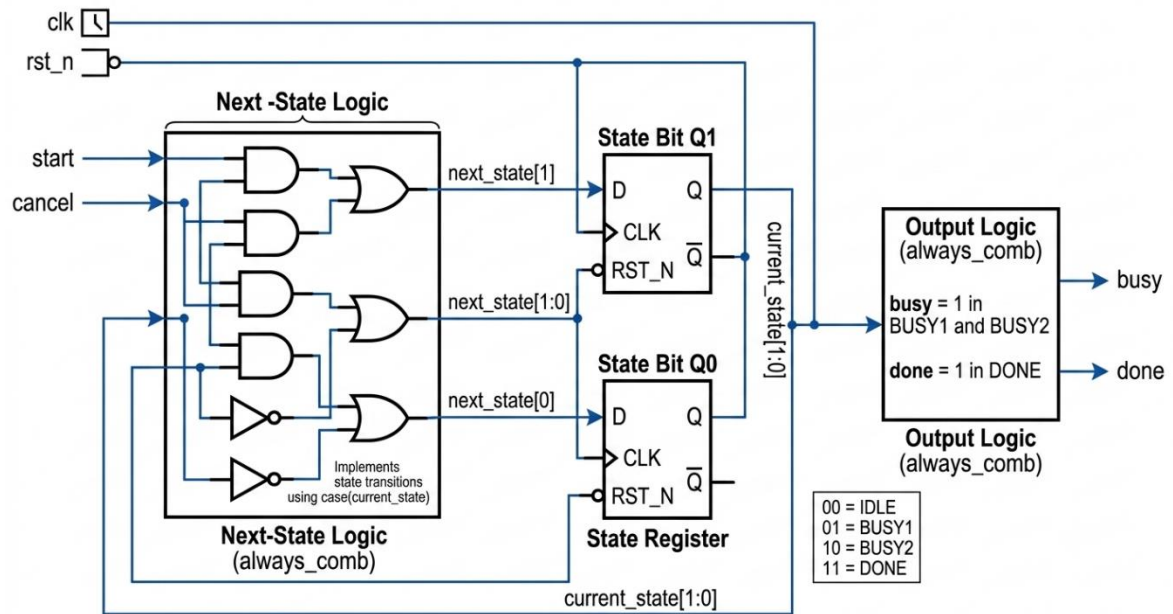
5. Simulation Output Screenshot:

```

Compiler version X-2025.06-SP1_Full164; Runtime version X-2025.06-SP1_Full164; Jul 29 08:44 2026
[PASS] Reset successful.
[PASS] Step=1 State=001
[PASS] Step=2 State=011
[PASS] Step=3 State=100
[PASS] Step=4 State=111
[PASS] Step=5 State=000
[PASS] Step=6 State=001
[PASS] Step=7 State=011
[PASS] Step=8 State=100
[PASS] Step=9 State=111
[PASS] Step=10 State=000
[PASS] Step=11 State=001
[PASS] Step=12 State=011
[PASS] Step=13 State=100
[PASS] Step=14 State=111
[PASS] Step=15 State=000
Simulation Completed

```


3. Block Diagram:



4. Simulation Output Screenshot:

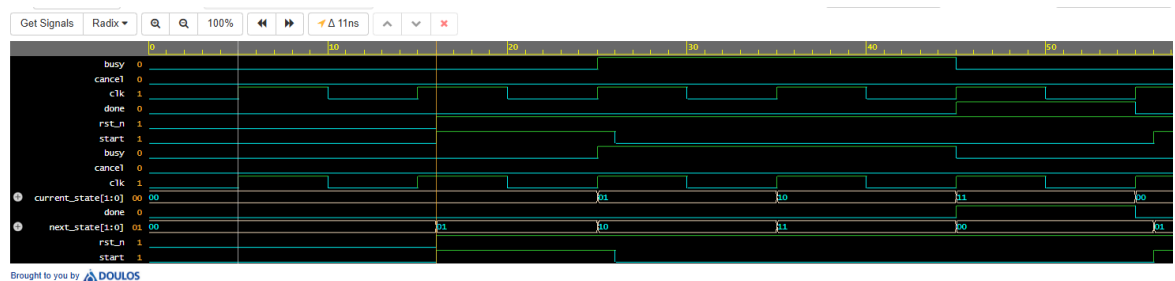
```

Compiler version X-2025.06-SP1_Full164; Runtime version X-2025.06-SP1_Full164;  Jul 31 02:51 2026
[PASS] Reset Successful
[PASS] BUSY1 State
[PASS] BUSY2 State
[PASS] DONE State
[PASS] Returned to IDLE

Test Case 2 : Cancel Operation
[PASS] BUSY1 State
[PASS] Cancel Returned to IDLE
$finish called from file "testbench.sv", line 122.
$finish at simulation time 76

VCS Simulation Report
Time: 76 ns
  
```

5. Waveform Screenshot:



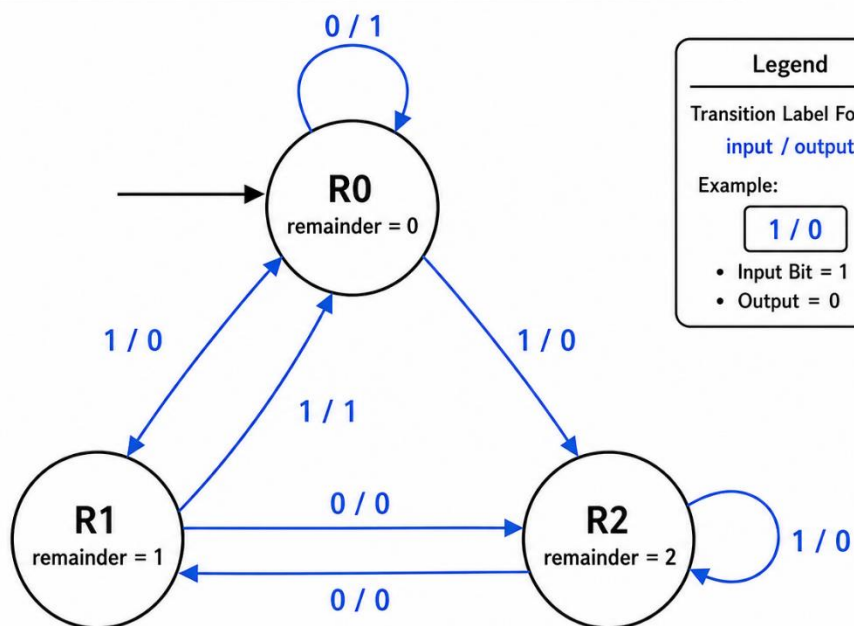
6. **Explanation:** This task implements a **Moore Finite State Machine (FSM)** with two input signals (`start` and `cancel`) and two output signals (`busy` and `done`) to model a simple request-processing controller. The FSM begins in the **IDLE** state and transitions to **BUSY1** when the `start` signal is asserted. It remains in the **BUSY1** and **BUSY2** states for a fixed processing duration while asserting `busy = 1`. After processing is complete, the FSM enters the **DONE** state, where `done = 1` is asserted for one clock cycle before returning to the **IDLE** state. If the `cancel` signal is asserted during either **BUSY** state, the controller immediately returns to the **IDLE** state without asserting the `done` signal.

The design follows the standard **3-block FSM coding style**, consisting of an `always_ff` block for the state register, an `always_comb` block for the next-state logic, and another `always_comb` block for the Moore output logic. A self-checking testbench verifies both the normal processing sequence (**IDLE** → **BUSY1** → **BUSY2** → **DONE** → **IDLE**) and the cancel sequence (**IDLE** → **BUSY1** → **IDLE**). The simulation results confirm that the controller correctly generates the `busy` and `done` outputs and performs the required state transitions according to the specification.

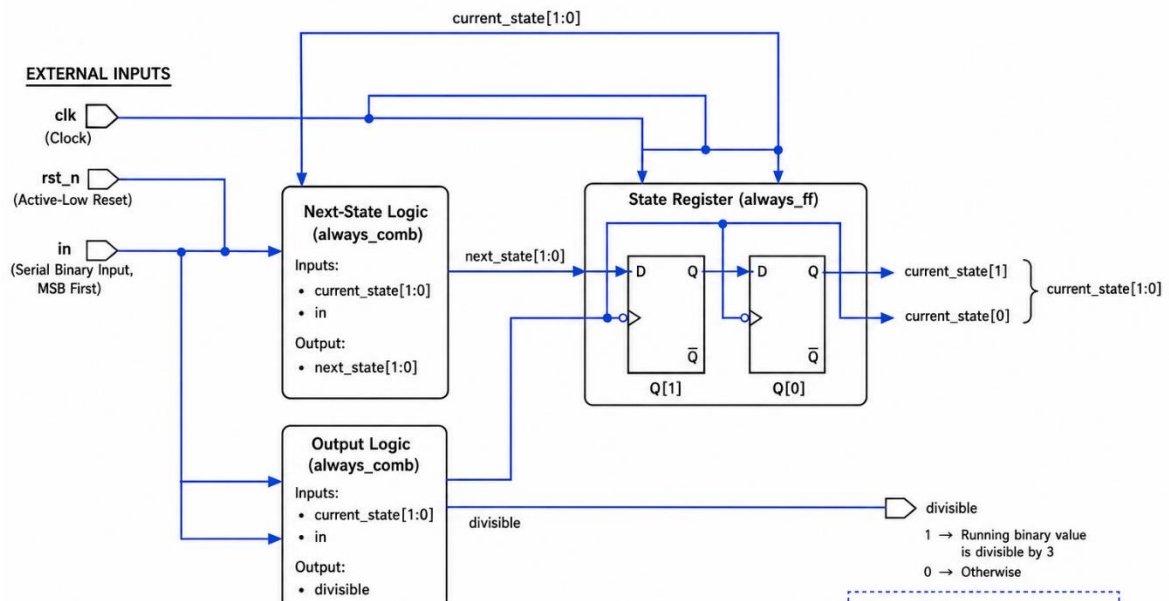
Task 16: Mealy FSM: Serial Binary Input Divisible by 3

7. **EDA Playground Link:** <https://edaplayground.com/x/LaME>

8. **State Diagram:**



9. Block Diagram:



10. Simulation Output Screenshot:

```
Compiler version X-2025.06-SP1_Full164; Runtime version X-2025.06-SP1_Full164; Jul 31 15:4
FSM Reset Complete
```

```
Test 1 : Binary 110 (Decimal 6)
Expected Outputs : 0 1 1
[PASS] Input=1 Output=0
[PASS] Input=1 Output=1
[PASS] Input=0 Output=1
FSM Reset Complete
```

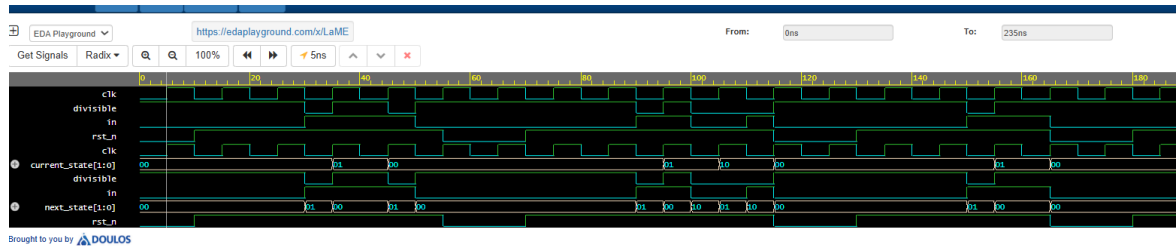
```
Test 2 : Binary 101 (Decimal 5)
Expected Outputs : 0 0 0
[PASS] Input=1 Output=0
[PASS] Input=0 Output=0
[PASS] Input=1 Output=0
FSM Reset Complete
```

```
Test 3 : Binary 11 (Decimal 3)
Expected Outputs : 0 1
[PASS] Input=1 Output=0
[PASS] Input=1 Output=1
FSM Reset Complete
```

```
Test 4 : Binary 1001 (Decimal 9)
Expected Outputs : 0 0 0 1
[PASS] Input=1 Output=0
[PASS] Input=0 Output=0
[PASS] Input=0 Output=0
[PASS] Input=1 Output=1
```

```
Simulation Completed Successfully.
$finish called from file "testbench.sv", line 117.
$finish at simulation time 235
```

11. Waveform Screenshot:



12. State Transition Table

Current State	Input (<i>i_n</i>)	Next State
R0	0	R0
R0	1	R1
R1	0	R2
R1	1	R0
R2	0	R1
R2	1	R2

13. Explanation: A Mealy FSM was designed to determine whether the running serial binary number is divisible by 3. Instead of storing the complete binary number, the FSM tracks only the remainder modulo 3 using three states: **R0 (remainder 0)**, **R1 (remainder 1)**, and **R2 (remainder 2)**. The next-state logic computes the new remainder based on the current state and the incoming serial bit, while the output logic asserts `divisible = 1` whenever the updated running value is exactly divisible by 3. The design follows the standard **3-block FSM coding style** using an `always_ff` block for the state register and two `always_comb` blocks for the next-state and output logic. This approach minimizes hardware by storing only the remainder instead of the entire binary number, making it efficient for FPGA and ASIC implementations.